Flyweight Pattern

The Flyweight Design Pattern is a structural design pattern that focuses on reducing the number of objects that need to be created, minimizing memory usage, and increasing performance. This pattern is especially useful when dealing with a large number of objects with similar states or configurations.

The key principle behind the Flyweight pattern is to separate the intrinsic state from the extrinsic state of an object:

Intrinsic State: This is the shared part of the state that is common across all objects and can be centralized. The intrinsic state is stored in the flyweights and is immutable.

Extrinsic State: This state varies between objects and cannot be shared. The client code must provide it when it uses the flyweight.

Flyweight objects are typically managed by a factory that ensures proper sharing. When a client requests a flyweight, the factory checks if an appropriate flyweight already exists and returns it; if not, it creates a new one.

Real World Example

A real-world example of the Flyweight pattern is seen in text editors that handle the formatting of characters in a document. If an editor did not use the Flyweight pattern, it would have to create a separate object for each character in the document, each storing its formatting information such as font size, style, and color. This would lead to a huge memory footprint, especially for large documents.

How Flyweight Pattern Applies:

- The intrinsic state includes the character itself and the font face, which is common and can be shared among many character instances in the document.
- The extrinsic state is the position of the character in the document, along with other unique formatting attributes, such as whether it's bold, italic, underlined, or highlighted.

Structure of Flyweight Pattern

The structural class diagram of the Flyweight Pattern contains the following components:

- **Flyweight Interface**:

  This is usually an abstract class or interface that contains methods that flyweight objects implement. It defines the method (`extrinsicState`) that the Concrete Flyweight is supposed to implement. The method receives the extrinsic state as an argument.

- **Concrete Flyweight**:

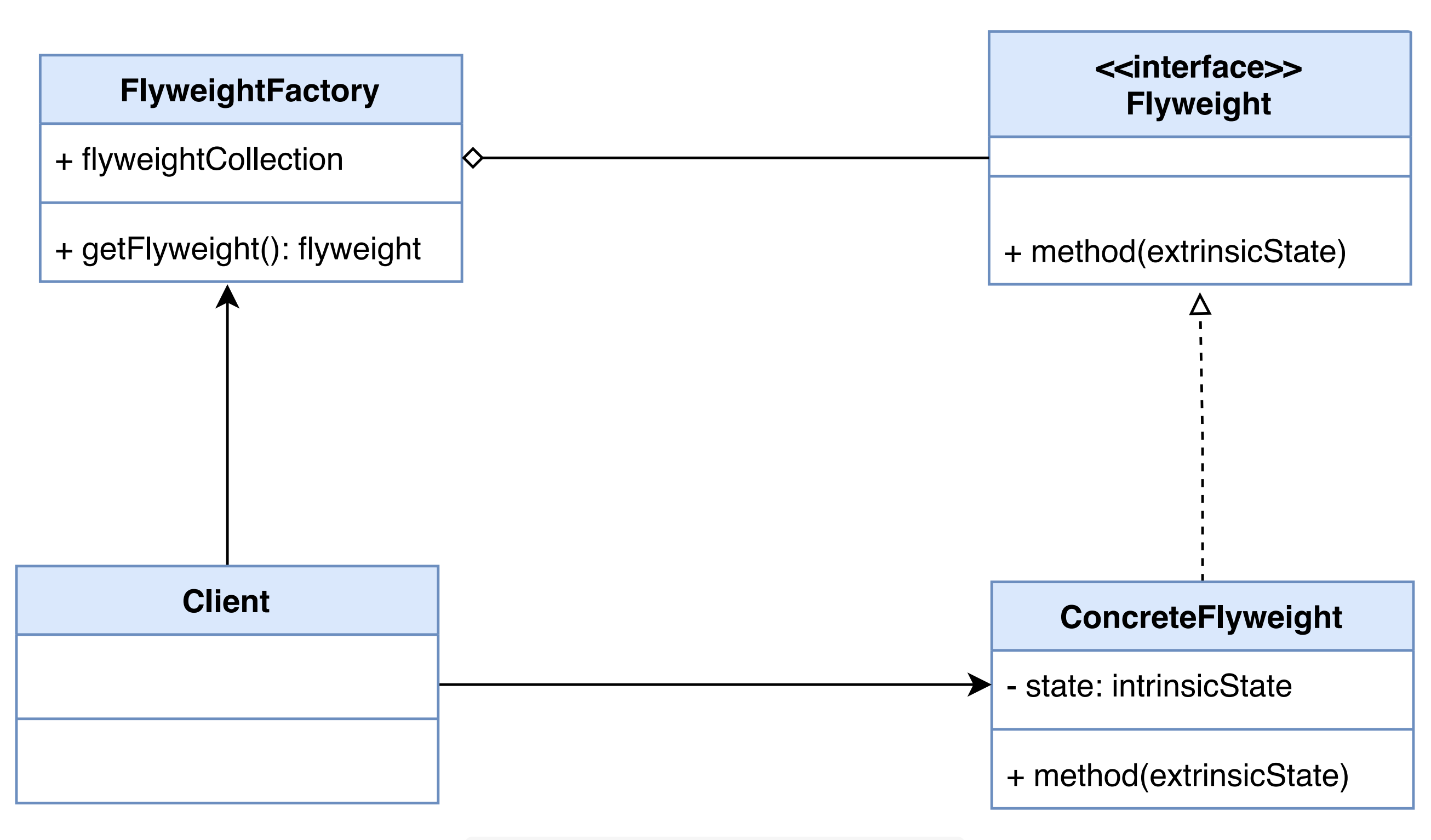
  Implements the Flyweight interface and has the ability to store shared intrinsic state for the application. The flyweight's internal state, or intrinsic state, usually remains constant once it is set. To carry out some operations, the method (`extrinsicState`) would make use of both the intrinsic and the given extrinsic state.

- **Flyweight Factory**:

  Responsible for creating and managing flyweight objects. It makes sure flyweights are shared rather than duplicated. It includes a `flyweightCollection` collection, that stores the flyweights that are currently in use. The `getFlyweight()` method first determines whether the flyweight already exists. If not, it creates a new one, adds it to the collection, and returns the current one.

- **Client**:

  The client keeps track of flyweight(s). To obtain the flyweight objects, it invokes the factory's `getFlyweight()` method. It calls the flyweight object's method (`extrinsicState`) when an operation needs to be performed, passing in the extrinsic state that is required.



Flyweight Pattern - Class Diagram

- The Client "uses" the Flyweight Factory to obtain flyweight objects.
- The Flyweight Factory "creates" flyweights.
- The Concrete Flyweight "implements" the Flyweight interface.

Implementation of Flyweight Pattern

The scenario:

Assume you are creating a particle system for a video game in which thousands of particles are rendered on screen, such as smoke, sparks, or magic effects. In addition to shared characteristics like texture, shape, and color, every particle possesses unique characteristics like position, velocity, and lifespan.

Flyweight Application:

Instead of storing texture, shape, and color for each particle, use the Flyweight Pattern to share these properties across all particles, significantly reducing the memory footprint.

The pseudocode for this example is as follows:

```
// Flyweight class for particle properties
CLASS ParticleType
    PRIVATE texture, shape, color
    PRIVATE STATIC typesCache = {}

    STATIC METHOD createType(texture, shape, color)
        IF NOT typesCache.hasKey(texture + shape + color)
            typesCache[texture + shape + color] = new ParticleType(texture, shape,
color)
        RETURN typesCache[texture + shape + color]

    CONSTRUCTOR(texture, shape, color)
        this.texture = texture
        this.shape = shape
        this.color = color

// Context class for individual particles
CLASS Particle
    PRIVATE x, y, velocityX, velocityY, lifespan, type

    CONSTRUCTOR(x, y, velocityX, velocityY, lifespan, type)
        this.x = x
        this.y = y
        this.velocityX = velocityX
        this.velocityY = velocityY
        this.lifespan = lifespan
        this.type = type

    METHOD update()
        // Update particle position and lifespan
        // ...

    METHOD draw()
        // Output the particle with its shared type properties
        PRINT "Drawing particle at (" + this.x + ", " + this.y + ") with texture: " +
this.type.texture

// Particle System class, acting as a client
CLASS ParticleSystem
    PRIVATE particles = []

    METHOD addParticle(x, y, velocityX, velocityY, lifespan, texture, shape, color)
        type = ParticleType.createType(texture, shape, color)
        particle = new Particle(x, y, velocityX, velocityY, lifespan, type)
        particles.add(particle)

    METHOD updateAndDraw()
        FOR particle IN particles
            particle.update()
            particle.draw()

// Client code
particleSystem = new ParticleSystem()
particleSystem.addParticle(0, 0, 1, 1, 60, "SmokeTexture", "Circle", "Gray")
particleSystem.addParticle(10, 10, 2, 2, 60, "SmokeTexture", "Circle", "Gray")
particleSystem.updateAndDraw()
```

- The `ParticleType` class is a flyweight that contains the shared properties for particles.
- `createType` manages a cache to reuse `ParticleType` instances with the same properties.
- `Particle` instances have unique properties like `position` and `velocity`, but share a common `ParticleType`.
- The `ParticleSystem` class manages a collection of particles. When a new particle is added, it uses the `ParticleType.createType` to get or create a shared type.
- During the game loop, each particle is updated and drawn using both its unique properties and shared type properties.

This example shows how the Flyweight Pattern can optimize a game's particle system, allowing for thousands of particles to be rendered without significant memory overhead.

Let's implement particle system example in programming languages.

Implementation

Java

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

class ParticleType {
    private static Map<String, ParticleType> cache = new HashMap<>();
    private String texture, shape, color;

    private ParticleType(String texture, String shape, String color) {
        this.texture = texture;
        this.shape = shape;
        this.color = color;
    }

    public static ParticleType getParticleType(String texture, String shape, String colo
        String key = texture + shape + color;
        if (!cache.containsKey(key)) {
            cache.put(key, new ParticleType(texture, shape, color));
        }
        return cache.get(key);
```

Run Code Result | Stdout

You must run your code first

Execute

Application of Flyweight Pattern

The Flyweight Pattern is especially useful in scenarios where the goal is to save memory by sharing as much data as possible between similar objects. Here are some specific applications where this pattern comes in handy:

- **Game development and graphics**:

  In games and graphics software, it is frequently necessary to render many similar objects. The system can share common properties between multiple objects, such as models, textures, or terrain data, by using flyweights. For instance, a few different tree models can be used to render a forest scene with thousands of trees.

- **Text Editors**:

  Word processors or text editors can use the Flyweight Pattern to control character formatting. The amount of memory used can be greatly decreased by sharing font, size, and style across characters with the same formatting, compared to each character having to store these attributes separately.

- **Designing User Interfaces**:

  Many times, UI frameworks have to produce many identical objects, like buttons, icons, or labels. It is possible to share UI element properties such as fonts, icons, and color schemes by using the Flyweight Pattern.

- **Networking**:

  Flyweights are useful in network applications for managing the context and state of connections, especially when working with many similar connections or sessions.

- **Database Applications**:

  Applications frequently need to represent many similar data objects when working with large datasets. Flyweights can manage shared data to minimize memory usage for cacheable data.

- **Development of Web Applications**:

  The Flyweight Pattern helps reduce the overall memory footprint of web pages by managing a shared style or behavior for a large number of similar DOM elements that are dynamically generated.

In each application, the flyweight serves as a shared object that can be used in multiple contexts, each with its specific state. By using the intrinsic (shared) state effectively, the Flyweight Pattern ensures that the memory usage is optimized without compromising the application's functionality or design.

Pros and Cons

Pros	Cons
Memory Saver: It's great at reducing memory usage.	**Complexity Alert**: It can make the system more complex.
Speed Booster: Less memory means faster performance.	**State Juggling**: Managing intrinsic and extrinsic states can be tricky.
Scalability King: It can handle scaling up like a champ.	**Not Always Clear**: For newcomers, the pattern might be a bit hard to grasp.

In short, Flyweight Pattern declutters your memory usage by sharing common data. It's super useful when you need to manage a lot of similar objects but don't want your memory crying for help. Just remember, it's a trade-off between memory efficiency and system complexity. Keep it in your toolkit for those memory-heavy scenarios!

← Previous
Facade Pattern

Next →
Proxy Pattern

☐ Mark as Completed